VerifRAG Technical Specification

Version: 1.0 | **Date:** 2/3/2026 | **Team:** Group X

Sean Collins (CS), Matthew Mankiewicz (CS), Luke Dolan (IT), Albert Hua (CS)

---

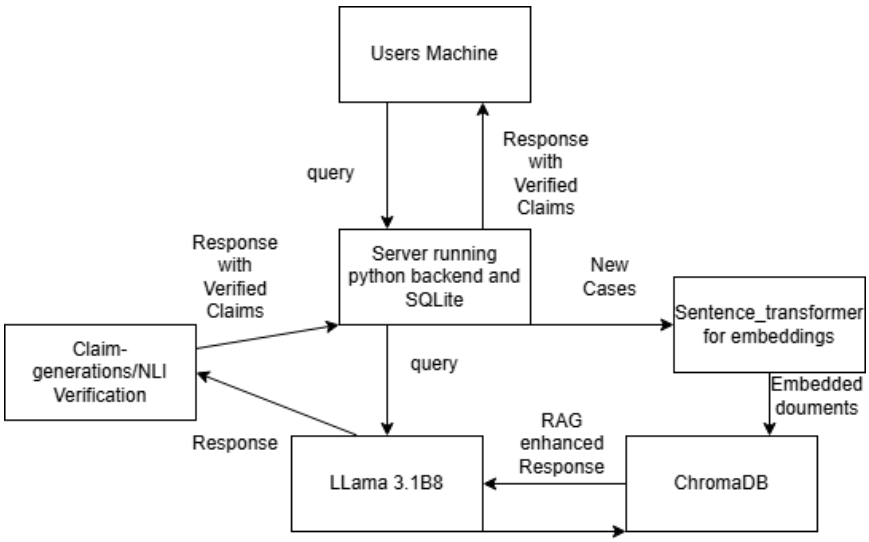
1. Product Overview

- **VerifRAG aims to reduce harm from hallucinations in LLMs tied to legal RAG systems.**
- **The primary market for any sort of legal RAG system will be paralegals working at mid to large-sized firms, as their jobs require constant interaction with the legal documents and research.**
- **Minimum Viable Product:**
  Feature A: Paralegals can upload, manage, and automatically encode legal documents into the vector store.
  Feature B: Paralegals can submit natural language queries.
  Feature C: VerifiRAG retrieves relevant context chunks via vector search and generates a response through a LLM.
  Feature D: VerifiRAG parses the generated response into atomic, sentence-level claims.
  Feature E: VerifiRAG uses a NLI model to strictly compare each claim against the retrieved context chunks for support or contradiction.
  Feature F: The UI displays the response with confidence indicators for each claim, to warn the user of potential hallucinations.

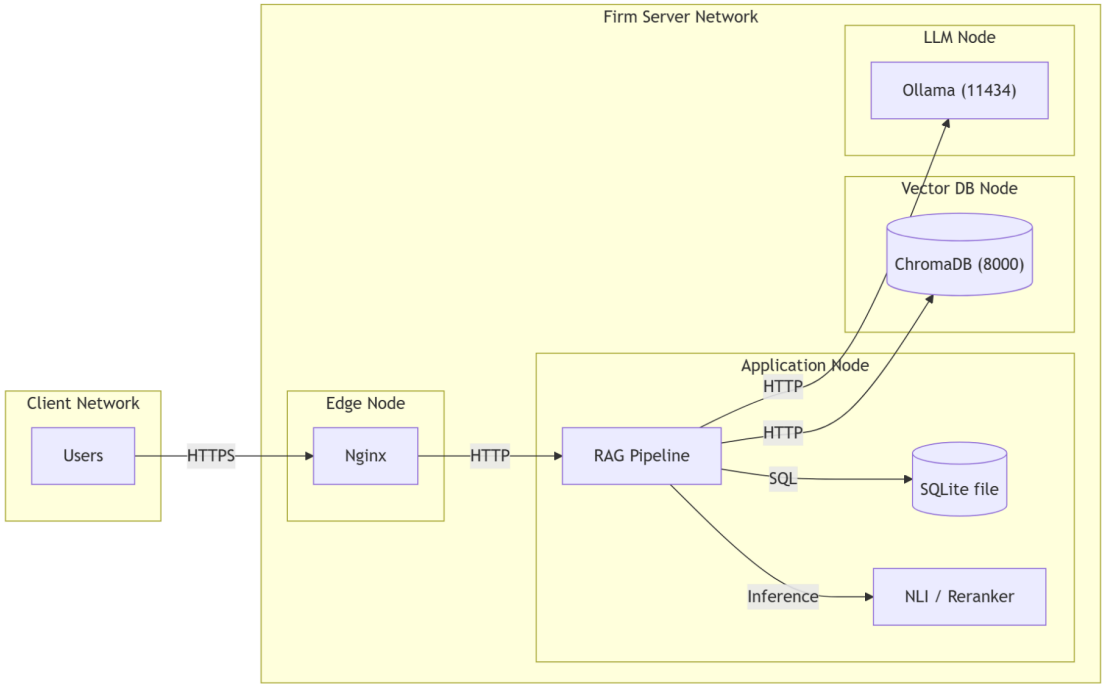
---

2. System Architecture (The "How")

- **Dataflow Diagram:**



Sever Architecture



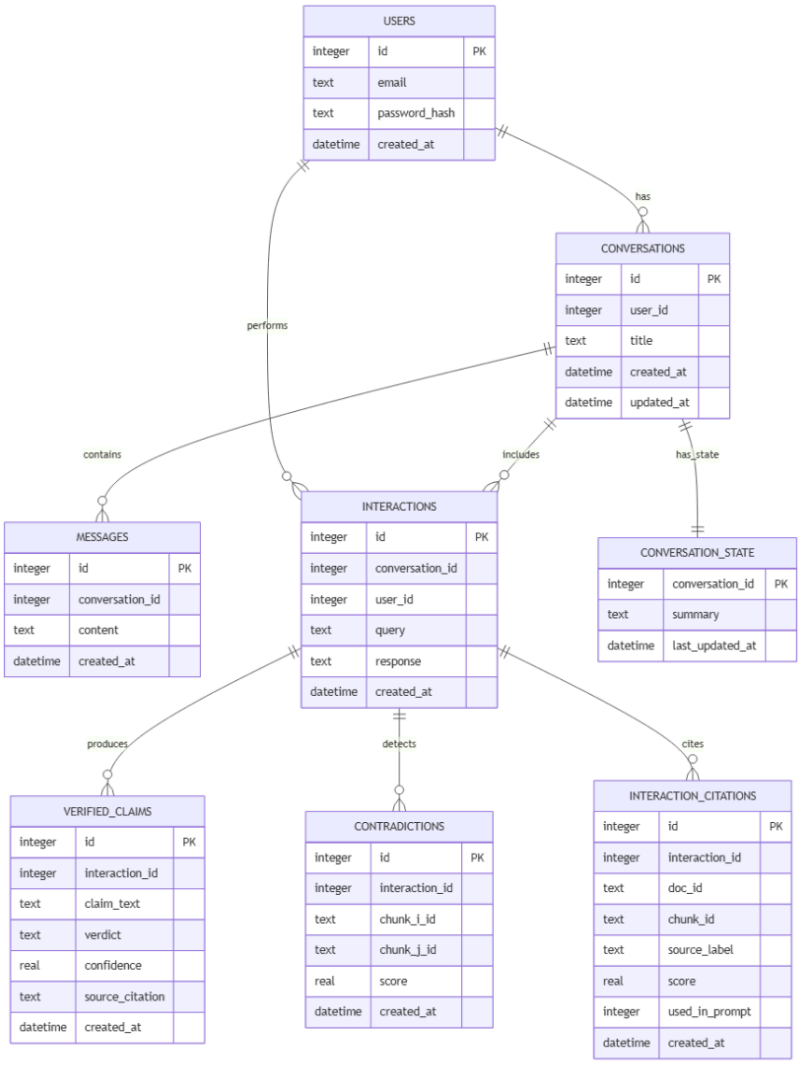
- **The Tech Stack:**
- **GUI:** We chose Streamlit because it allows our team to build the entire tool in Python. This saves us from having to learn and maintain a separate language. For a law firm, this is better because the app runs over a private local network, making it easier for their IT team to secure compared to a standard web app.
- **Logic Processing:** Python is the industry standard for AI and provides the specialized libraries required for legal document processing. PyMuPDF will be used to handle the complex, multi-layered PDFs common in court filings, and python-docx allows paralegals to upload internal drafts and correspondence directly from Microsoft Word. spaCy will be used for chunking, which breaks documents down into “atomic" units like specific clauses or sentences rather than arbitrary character counts.
- **Vector Database:** We chose ChromaDB because it is the most popular open-source tool for storing and searching embeddings. For a mid-to-large law firm, Chroma is reasonable to maintain because it can be run entirely on a local server without needing a cloud subscription.
- **Large Language Model:** To adhere to ABA Rule 1.6 (Attorney-Client Privilege), it is ideal to keep client data off of the cloud. By running Llama locally on the firm's hardware, we guarantee that confidential documents never leave the firm’s network.
- **Relational Database:** SQLite was chosen because it is a "zero-setup" database - it stores all user login info in a single local file. Since we only need to store a few user accounts per office, along with their conversations, SQLite is the fastest and most reliable choice.
- **Verification Engine:** We chose DeBERTa to handle the "fact-checking" part of our project. While general LLMs are suitable for generating human-like responses, they are

based on prediction and can often make mistakes. DeBERTa is a specialist model designed for NLI, and running it enables us to check every generated claim against legal documents to successfully flag contradictions or hallucinations.

- **Justification of Technology**
- We selected a ‘local-first’ stack to address the two most significant hurdles mid-to-large law firms face: data security and cost. Under ABA Model Rule 1.6 lawyers must protect client secrets, and by keeping the model and document database entirely on the firm’s own hardware, we ensure that sensitive case files remain within the bounds of the firm. We shifted our target consumer base to these firms because they likely have hardware meeting or exceeding the minimum system requirements needed to run these tools locally, and they also most likely employ an IT staff/division to manage this hardware. They are also much more likely to employ a team of paralegals compared to small firms and independent practitioners who lack the budget or manpower to do so.
- By utilizing existing infrastructure, firms can also avoid the heavy supplementary costs that are usually associated with legal AI tools, such as monthly per-user subscription fees or expensive API tokens that can skyrocket costs. Because our system runs on the firm's current workstations and servers, the primary investment is the initial setup rather than an endless cycle of billing, which makes VerifiRAG an attractive ‘one-time’ value purchase that scales with a firm’s existing IT budget rather than competing against it.
- Likely due to the scale of organization we are targeting, around 100 members, sqlite will be sufficient, and will simplify the process at this scale.

---

3. Data Design



- **Core Entities**: User, Conversation, Message
- **Data Schema**: Users are connected to the system, and they have conversations that are composed of messages. Interactions between the user and the LLM are stored with connections to clarify things like claims, contradictions, and the citations that make up the message.
- **Storage**: Text and vector embeddings are stored locally using ChromaDB. We chose ChromaDB's persistent local storage over cloud options to guarantee that sensitive legal data never leaves a firm's physical hardware. Firms can scale the database as large as their local disk space allows. The users and messages will be stored on a sqlite file.

---

4. Technical Complexity & Depth

- **Retrieval and Context Assembly:** Legal documents are retrieved from validated sources and user-uploaded files, filtered by jurisdiction and recency, and then ranked by legal authority (statutes > cases > commentary, and so on). PyMuPDF and python-docx are used to parse uploaded documents, external APIs provide statutory and case law data, and ChromaDB enables dense vector retrieval.
- **Claim Generation and Decomposition:** A locally hosted model generates responses, which are broken into sentence and clause level claims parsed through Python. Claims are then tagged with topic metadata and structured into a claim tree.
- **Verification and Contradiction Detection:** Natural Language Inference (NLI) models are used to detect contradictions and inconsistencies across sources, with clustering techniques implemented to identify potentially conflicting interpretations of claims.
- **Confidence Scoring and Presentation:** Claims are given a score based on the number and authority of supporting sources, consistency across documents, and recency of law, with penalties for contradictory or weak evidence. Results are presented with confidence indicators, verified citations, and warnings for each individual claim.

---

5. Functional Requirements

- **A. Users & Access**

  1. User registration/login/logout: The system shall allow users to create an account, authenticate, and end a session.

  2. User session isolation: The system shall ensure each user can only view their own queries, responses, uploaded documents, and verification results.

  3. User profile/basic settings: The system shall allow users to view and update basic profile info.

- **B. Document Ingestion & Management**

  1. Upload legal documents: The system shall allow users to upload PDF and DOCX files through the UI.

  2. Document validation: The system shall validate file type, size limits, and reject corrupted/unsupported files.

  3. Document parsing: The system shall extract text from uploaded files (PDF via PyMuPDF, DOCX via python-docx).

  4. Chunking: The system shall split extracted text into chunks suitable for retrieval (sentence/paragraph-based, with offsets and page references where possible).

5. Indexing in ChromaDB: The system shall generate embeddings for chunks and insert them into ChromaDB with filterable metadata.

6. Delete/replace documents: The system shall allow users to delete a document and its associated vectors; and re-upload a new version.

- **C. Retrieval and Context Assembly**

  1. Submit a query: The system shall allow users to enter a natural-language research question.

  2. Filter retrieval: The system shall support retrieval filters at query time (at minimum: jurisdiction, date range, and doc type).

  3. Vector search: The system shall retrieve the top-K most relevant chunks from ChromaDB for a query.

  4. Hybrid retrieval (optional per spec): The system shall support lexical search (BM25) and combine results with dense retrieval when enabled.

  5. Authority-aware ranking: The system shall rank or rerank retrieved results by legal authority (e.g., statutes > cases > commentary) in addition to similarity.

  6. Context packaging: The system shall assemble a context window from retrieved chunks within model token limits and track which chunks were used.

- **D. Response Generation**

  1. Generate an answer: The system shall generate a response using the configured LLM based on the retrieved context.

  2. Citations in response: The system shall attach citations to supporting text, linking sentences/claims to source chunks with page/section info when available.

  3. Response history: The system shall store and display prior queries and generated responses per user.

- **E. Claim Decomposition**

  1. Extract claims: The system shall decompose the generated response into sentence-level and/or clause-level claims.

  2. Claim structuring: The system shall assign each claim an ID/order and optionally build a claim tree (parent/child relationships).

  3. Claim metadata tagging: The system shall tag claims with basic topic labels

(rule/definition/holding/procedure/etc. or simple keywords).

- **F. Claim Verification**

  1. Evidence retrieval per claim: The system shall retrieve supporting and potentially contradicting chunks for each claim (may reuse initial retrieval or run per-claim retrieval).

  2. NLI contradiction detection: The system shall run an NLI model to label claim-evidence relationships as entailment/contradiction/neutral.

  3. Cross-claim consistency checks: The system shall detect contradictions between claims in the same response using NLI (claim vs claim).

  4. Per-claim confidence score: The system shall compute a confidence score per claim based on:

  5. Claim-level warnings: The system shall flag claims as “verified / weakly supported / contradicted / insufficient evidence” with explanations.

  6. Clustering for conflicting interpretations (optional): The system shall cluster similar claims/evidence to surface conflicting interpretations.

- **G. UI**

  1. Upload UI: The UI shall provide a document upload workflow with progress/status.

  2. Query UI: The UI shall provide a query input, retrieval filter controls, and a “run” action.

  3. Results UI: The UI shall display: the generated response, list of claims, confidence indicators, citations, contradiction warnings.

  4. Evidence drill-down: The UI shall allow users to open the source chunk details (doc title, page, excerpt) used as evidence.

- **H. Logging, Controls, and Safety**

  1. Request timeout/escape hatch: The system shall allow users to cancel a long-running request and enforce time thresholds for ingestion, retrieval, and verification steps.

  2. Input Validation: The system shall sanitize/normalize user inputs and documents (e.g., strip hidden instructions, constrain system prompt) to reduce prompt injection risk.

  3. Audit trail: The system shall log key events (upload/indexing, retrieval run, model call,

verification run) for debugging and evaluation.

- **I. External Sources:**

  1. External Legal Input Ingestion: The system shall ingest documents from CourtListener (opinions/dockets), GovInfo/GPO (U.S. Code/CFR), and the Caselaw Access Project (historical corpus) to build a local reference library via their respective APIs and bulk data tools.

  2. Source Display: The system shall visually distinguish between user-uploaded files (Private Case Data) and the pre-loaded legal base set (Validated Primary Authority) to ensure the user knows the weight of each cited source.

---

6. Non-Functional Requirements

- **A. Accuracy & Reliability**

  1. Verification Threshold (F1 Score): The DeBERTa (NLI) component shall achieve an F1 score of at least 0.80, which indicates a 80% success rate in catching contradictions.

  2. Citation Integrity: 100% of the citations displayed in the UI must link to a valid source chunk within the local ChromaDB index. This prevents "phantom citations" while allowing the system to occasionally flag a source as "insufficient" if the law is unclear.

  3. Consistency: For identical queries against the same document set, the system shall provide reproducible results. We allow for a small variance (~5%) in confidence scores to account for the probabilistic nature of local LLM inference.

- **B. Performance & Latency**

  1. Response Generation Time: The system shall produce a complete response and verification report within 90 to 120 seconds for standard queries. This window accounts for the local GPU processing power required to run Llama and DeBERTa back-to-back.

  2. Ingestion Speed: The PyMuPDF and python-docx parsers shall process a typical 25-page legal filing in under 20 seconds. This ensures the UI remains responsive during the initial setup of a case.

  3. Search Latency: Local ChromaDB searches must complete in under 250ms.

- **C. Usability (UX)**

1. Traffic Light Logic: The UI shall use simple color-coding (Red/Yellow/Green) to represent verification results. This reduces the complex math of the NLI model into a format that a paralegal can quickly check and understand in seconds.

2. Activity Indicators: Because local processing takes time, the Streamlit UI must show a visible "Processing" spinner or progress bar. This tells the user the system is working and hasn't crashed.

- **D. Hardware & Resource Optimization**

  1. VRAM Management: The system shall be optimized to run on a single consumer-grade GPU with at least 12GB of VRAM. If memory runs low, the system should prioritize the Verification (DeBERTa) model to ensure accuracy over generation speed.

  2. Disk Usage: The local ChromaDB and SQLite files should not exceed 2GB of storage for a typical case file (~1,000 documents). This ensures the software can run on standard office laptops without requiring a server upgrade.

---

7. Error Handling & Resilience

- **Local Resource Exhaustion (GPU/VRAM)**
  - **The Risk:** The local GPU could run out of memory while running Llama and DeBERTa at the same time.
  - **The Handling:** The system shall monitor VRAM usage. If a memory spike is detected, the system will pause the Verification step and notify the user with a message, something along the lines of: “Local resources are full, please close other applications or wait for the system to clear memory before continuing". This prevents the entire app from crashing.
- **Document Parsing Failures**
  - **The Risk:** A legal PDF might be password-protected, corrupted, or formatted in a way that PyMuPDF cannot read.
  - **The Handling:** UI will flag the specific file in the ingestion list with a "Retry/Skip" option. The system will log the error so the paralegal knows to upload a clean, unencrypted version.
- **Model Timeouts**
  - **The Risk:** Occasionally, an LLM might get stuck in a loop or take too long to generate a complex legal argument.
  - **The Handling:** The system implements a Progressive Timeout. If a response takes longer than 90 seconds, a "Stop Generation" button will appear in the

Streamlit UI. This allows the user to kill the current process and refine their query without needing to restart the entire software.

- **NLI Ambiguity (Verification Failure)**
    - **The Risk:** The DeBERTa model might return a "Neutral" score if it can't find a clear link between a claim and the evidence.
    - **The Handling:** Instead of guessing, the system will flag the claim with a Yellow (Insufficient Evidence) warning. It will explicitly tell the user that there was not a clear match found within the documents, and will instruct the paralegal to manually verify the claim, which is the safest legal approach.